

Structure chart and Functions (In-class exercise)

Overview: We will decompose a problem into a hierarchy of modules (structure chart), then translate that hierarchy directly into Python functions with parameters and return values.

Scenario:

The Murray Print Shop wants a simple program to quote the cost of a print job before a customer commits to it. A staff member enters the job details, and the program should calculate and display a full price breakdown. A customer may submit multiple jobs in one order.

Here's how pricing works:

- **Base cost per page** is \$0.10 for black-and-white, \$0.50 for color.
- **Binding** is optional: spiral binding adds a flat \$2.00; no binding costs nothing. Most students do not opt for binding.
- **Rush orders** (needed same day) add a 25% surcharge to the base printing cost only, calculated *before* binding is added.
- **Student discount:** if the customer has a valid student ID, they get 10% off the *total* (printing + rush + binding).
- The program should display an itemized receipt: printing cost, rush surcharge (if any), binding cost (if any), discount applied (if any), and the final total.

Task 1 — Structure Chart

Draw a structure chart that decomposes the program into modules. Show dataflow in the structure chart.

Task 2 - Translate the chart into functions

Use the functions_exercise.py to Implement each box as a function. Verify the program works as expected.

Here is a sample data and results

Costs									
BW	\$0.10								
Color	\$0.50								
Binding	\$2								
Rush	25%								
Discount	20%								
Order_job	Job	BW	Color	Binding	Printing co	Binding cost			
1	1	10	0	Yes	\$1.00	\$ 2.00			
1	2	0	4	No	\$2.00	\$ -			
2	1	20	3	No	\$3.50	\$ -			
Orders	Discount	Rush	Print cost	Rush char	Binding	Total	Discount	Final total	
1	Yes	Yes	\$ 3.00	3.75	2	\$ 8.75	1.75	\$ 7.00	
2	No	No	\$ 3.50	3.5	0	\$ 7.00	0	\$ 7.00	